KHUFU

FREE GUIDE

The 7-Day Playbook

How a SaaS or mobile V1 actually gets scoped, built and shipped in seven days — the full working method, including the five things that blow it up.

khufu.io · Adrien De Coster · Updated 2026-07-31

What this is

Most V1s do not take six months because they are hard. They take six months because nobody ever forced a decision about what the product is not.

Seven days is not a stunt. It is a constraint that makes those decisions unavoidable. When the deadline cannot move, scope becomes the only variable — and scope is the thing you should have been cutting all along.

This is the method I use, unchanged, on client sprints and on my own products. Nothing here is withheld. If you read it and run it yourself, that is a fine outcome.

Who this is for

Founders who need a real product in front of real users in weeks, not quarters — and anyone about to pay someone else to build one.

What's inside

CHAPTER 01

The one rule: the deadline is fixed, the scope is the variable

Why a hard deadline is the only thing that reliably produces a shippable product.

Every project has four levers: scope, time, cost and quality. Traditional projects fix scope and let time and cost float. That is why they slip — there is no forcing function, so every "small addition" gets absorbed by the schedule until the schedule stops meaning anything.

A 7-day sprint inverts it. Time is fixed. Cost is fixed. Quality is non-negotiable, because a V1 that cannot be maintained is not a V1, it is a demo with a bill attached. That leaves scope as the only thing that can move — so scope is what gets discussed, honestly, on day one, instead of quietly expanding for four months.

The practical consequence

Once the deadline is real, "can we also add..." stops being a free question. It becomes "what comes out so this can go in?" That single change in framing is worth more than any tooling.

This is also why the price is fixed. A day rate rewards the builder for taking longer. A fixed price puts the cost of an over-run on the person who controls the estimate. If the sprint runs long, that is my problem, not a change order.

What a V1 is, and what it is not

A V1 is the smallest thing that does one job end to end, for a real user, in production. It is not a prototype, not a clickable mockup, and not an MVP in the degraded sense the word has acquired — a throwaway built on tooling you will have to escape later.

- It runs on a production stack, on your infrastructure, with your accounts.
- One user journey works completely — signup to the moment of value, with nothing faked in between.
- It can be deployed again tomorrow by someone who is not the person who built it.
- It is small enough that you could throw away half of it without grief, because you will.

CHAPTER 02

Day 0 — the scoping conversation

Five questions that turn a vague idea into a buildable one-sentence product definition.

The sprint is won or lost before it starts. Day 0 is a single conversation whose only job is to produce a sentence that both sides agree on. Not a spec document — a sentence.

The five questions

1. Who is the one user this is for? Not a segment — one person you can name or describe in a sentence. If there are two, pick the one who pays.
2. What is the one job they hire this product to do? One verb. "Track", "book", "compare", "publish". If you need "and", you have two products.
3. What do they do today instead? A spreadsheet, a WhatsApp group, a competitor, nothing. This tells you what "better" has to mean, and how high the bar actually is.
4. What is the single moment where they get the value? Find the screen or action where the user goes "oh, good". Everything in the V1 exists to get them to that moment; everything else is a candidate for the cut list.
5. How will you know in 30 days whether this worked? A number, however crude: signups, first paid customer, a demo that closes. If nobody can answer this, the product is not the problem — the bet is.

Answering those five properly takes about forty minutes. Most people have never been made to do it, which is why the answers are usually uncomfortable and always useful.

The output: a one-page scope contract

The conversation ends with one page, signed off before any code is written. It has four parts, and the fourth is the one that matters.

- The product sentence: "For [user], this lets them [one job] so that [outcome]."
- The critical flow: the ordered list of screens and actions from arrival to the value moment. Usually five to nine steps.
- The stack and where it runs, named explicitly, with whose accounts hold what.
- The cut list: everything discussed and deliberately not built, written down by name.

Write the cut list down

An unwritten "we agreed not to do that" evaporates by Wednesday. A written cut list turns a scope fight into a five-second lookup — and it becomes the roadmap for what comes after launch, which makes it useful rather than punitive.

CHAPTER 03

Days 1 to 7 — what happens, and what must be true at the end of each

The day-by-day breakdown with the exit condition for every single day.

Each day has an exit condition. If a day ends without its condition met, the problem is surfaced that evening rather than discovered on Friday. Nothing here is a status update for its own sake — a day that ends red gets scope removed the same night.

Day 1 — Scoping and architecture

The scope contract is finalised, the data model is drawn, the repository exists, the environments exist, and the deployment pipeline runs on an empty application. Deploying on day one sounds premature; it is the opposite. The riskiest, most surprise-prone part of any project is the first successful production deploy, and doing it while the app is empty means it costs an hour instead of a day.

Exit condition: an empty app is live on a real URL, and the data model fits on one page.

Days 2 and 3 — The core build

The critical flow gets built end to end, in order, with nothing stubbed. Authentication, the primary data objects, the screens the user actually passes through. The rule is depth before breadth: one complete flow beats six half-built ones, because a complete flow can be tested by a human and a half-built one cannot be tested by anyone.

Exit condition: at the end of day 3, someone who has never seen the product can complete the critical flow, on the deployed URL, without help.

Day 4 — Infrastructure, tests and security

The unglamorous day, and the one that separates a product from a demo. Environment variables and secrets handled properly. Database backups on. Error tracking and analytics wired in. Rate limiting where it matters. Automated tests on the paths where a failure is expensive — payment, auth, data writes — and not on the paths where they are theatre.

- Auth flows: signup, login, reset, session expiry.
- Anything touching money, before anything touching pixels.
- Destructive actions: delete, overwrite, bulk operations.
- A restore actually tested from a backup, not just a backup that exists.

Exit condition: you could hand the repository to another developer and they could run it locally from

the README, and a production error would page someone rather than vanish.

Days 5 and 6 — Iteration on real feedback

The product is in your hands and changing daily. This is where the founder earns the outcome: fast, specific feedback beats a long list delivered on the last day. The best sprints have a founder who checks the deployed URL twice a day and sends three sharp notes each time.

Feedback that works: "the empty state after signup does not tell me what to do next". Feedback that does not: "it feels a bit clunky". If a note cannot be turned into a change to a specific screen, it is a conversation, not a ticket — and conversations happen live, not in a document.

Exit condition: at the end of day 6 there is nothing in the critical flow you would be embarrassed to show a customer.

Day 7 — Ship and hand over

Production deployment on the real domain, CI/CD running, monitoring on, and the handover: repository access, infrastructure accounts, environment variables documented, a README that works, and a walkthrough recorded or live.

Exit condition: your V1 is online under your domain, in your accounts, and you could fire me on day 8 without losing anything but momentum.

Mobile apps: add store review

The seven days cover design, build and deployment. App Store and Google Play review time sits outside anyone's control — typically one to three days, occasionally longer on a first submission. Plan the launch date accordingly, and submit on day 6, not day 7.

CHAPTER 04

What AI actually does here — and what it does not

An honest accounting of where the speed comes from, minus the hype.

AI-native is a real operating change, not a badge. But the speed comes from a specific set of places, and being precise about them matters more than the label.

Where it genuinely compresses time

- Scaffolding: schemas, migrations, CRUD layers, forms, typed API clients. Work that is mechanical and voluminous, where a senior developer adds judgement only at the edges.
- Tests: generating the boring coverage around a well-specified function, which is exactly the work that gets skipped under deadline pressure.
- Translation and content plumbing: ten locales of interface copy is a week of coordination by hand, and an afternoon with review.
- Reading unfamiliar code and documentation fast — the difference between an hour and a day when integrating an unfamiliar API.

Where it does not help, and pretending otherwise is how projects fail

- Deciding what to build. Nothing removes the need for the day 0 conversation.
- Architecture that will still hold at ten thousand users. A model will happily produce a design that works today and traps you in six months.
- Anything requiring judgement about your business, your users or your risk tolerance.
- Debugging subtle production behaviour — race conditions, cache invalidation, the things that only appear under real load.

The honest ratio: on a typical V1, AI removes roughly a third to a half of the mechanical work. It removes none of the thinking. A sprint that is fast because the thinking was skipped produces a codebase you pay for twice.

> *The speed is not the AI. The speed is a fixed deadline, a written cut list, and one person who can hold the whole product in their head — with AI removing the typing.*

CHAPTER 05

The five things that blow up a 7-day sprint

The real failure modes, each with the mitigation that has to happen before day 1.

Sprints do not usually fail on the code. They fail on five things, and four of them are external. Each has a mitigation that costs nothing if it happens before the sprint starts, and costs days if it does not.

1. Third-party access that takes longer than the sprint

Payment provider onboarding, bank verification, an API partner whose approval flow runs on a weekly meeting, a government data source, an Apple developer account that needs a company registration number. Any one of these can take longer than the entire build.

Mitigation: list every external system on day 0 and start every account and approval process immediately — before the sprint begins, not on the day the integration is scheduled. Build against a sandbox or a mock behind a clean interface, and swap when access lands.

2. The undecided founder

The single most expensive failure mode. If a decision needs a co-founder who is travelling, a board that meets on Thursdays, or three days of deliberation, the sprint stalls on a question that takes ninety seconds to answer.

Mitigation: name one decision-maker with actual authority, agree they are reachable within four hours during the sprint, and agree the default — if a decision is not made within four hours, the simpler option ships and can be changed later.

3. Scope that arrives disguised as a detail

"Just add a role for managers." That is a permissions system. "It should work in Spanish too." That is an internationalisation layer touching every screen. Scope rarely arrives labelled as scope; it arrives as a reasonable-sounding small request on day 4.

Mitigation: the written cut list, plus one honest sentence when it happens — "yes, and here is what comes out". Not a refusal. A trade.

4. Content and data nobody owns

The product needs real text, real categories, a real product catalogue, a logo, legal pages. It is nobody's job, so it arrives on day 6 in a spreadsheet with the wrong structure, and the last two days become data cleanup instead of product work.

Mitigation: assign an owner and a deadline of day 3 for every piece of content and data, in the format that has been agreed. Placeholder content ships if real content is late — the deadline does not move for a logo.

5. Reviewing the wrong thing

A founder who spends days 5 and 6 on colours and copy while the critical flow has a broken edge case has spent the sprint's most valuable feedback window on the cheapest thing to change later.

Mitigation: review in the order of what is expensive to change. Data model and flows first, interaction second, visual polish last. Colours are a fifteen-minute change forever. A wrong data model is a rewrite.

CHAPTER 06

What you own on day 8

The handover checklist — what "you own it" has to mean concretely.

Ownership is either concrete or it is marketing. Concretely, at the end of a sprint you should hold all of the following, in your name, without asking anyone.

- The repository, on your organisation account, with full history. Not a zip file, not a fork of an agency template with proprietary parts.
- Every infrastructure account in your name and paid by your card: hosting, database, email, storage, analytics, error tracking.
- Your domain and DNS, under your registrar account.
- Environment variables documented, and secrets held by you.
- A README that a competent developer can follow to run the project locally in under thirty minutes.
- The design source files, if there were any.

The test is simple, and worth applying to any provider you are considering: if you ended the relationship the day after launch, what stops working? The correct answer is nothing except further changes.

That is also the reason the stack matters. A V1 built on real code — Next.js, NestJS, PostgreSQL, React Native — can be handed to any competent developer in any market. A V1 built on a proprietary builder can only be maintained by people who use that builder, at whatever price the platform decides later.

CHAPTER 07

Run it on your own idea — the one-hour version

A worked exercise: go from idea to scope contract in an hour, on your own.

You do not need me to do this part. Set aside an hour, take the idea you have been circling for months, and produce the one page.

Minutes 0 to 20 — the five questions

Write the answers as full sentences, not bullets. Sentences expose vagueness; bullets hide it. If question two needs an "and", split the product in two and pick one.

Minutes 20 to 40 — the critical flow

List every screen and action from a user's arrival to the value moment, in order. Number them. If you end up with more than twelve steps, the V1 is too big — find the step where value could arrive earlier and cut everything after it into version two.

Minutes 40 to 55 — the cut list

Now write down everything you thought of and are not building. Admin dashboards, roles and permissions, notifications, onboarding tours, settings pages, integrations, dark mode, an export feature, a mobile app if the web works. Be ruthless: most of this list is real work that no first user will notice is missing.

Minutes 55 to 60 — the 30-day number

One metric, one target, one date. "Ten paying users by 15 September." Write it at the top of the page, above everything else, because it is the only line that tells you whether any of the rest was worth building.

What to do with the page

Whatever you decide next — build it yourself, hire a freelancer, or hand it to me — that page is what makes any of those cheaper and faster. It is worth an hour of your time even if we never speak.

Questions

Is seven days realistic for a real product?

Yes, for a correctly scoped V1 — one user, one job, one complete flow, on a production stack. It is not realistic for a product with several user roles, a complex permissions model, heavy migrations from an existing system, or regulatory certification. The scoping conversation exists to tell you which of those you have, before anyone commits. Khufu's Sprint V1 is a fixed price of $17,000 (€15,000) for 7 days.

What happens if the sprint runs over?

It is a fixed-price engagement, so an over-run is the agency's cost, not a change order. That is precisely why the scope conversation on day 0 is thorough: the incentive to be honest about what fits sits with the person doing the estimating.

Who actually writes the code?

Adrien De Coster, the founder, on every sprint — a solo operator with AI tooling rather than a team with handoffs. No account managers, no juniors learning on your project, no work passed to a subcontractor you never meet.

What stack does a Khufu V1 run on?

Next.js and React for web, React Native and Expo for mobile, NestJS and Prisma with PostgreSQL for back ends, deployed on managed cloud infrastructure. Real code, in your repository, on your accounts — hand it to any developer on day one.

Can I use this playbook without hiring Khufu?

Yes, and that is the intent. The method is the method whether you run it with an in-house developer, a freelancer or yourself. If reading it means you scope your own build better and never contact me, the guide did its job.

Want this run on your product?

Sprint V1: a SaaS or mobile app scoped, built and shipped in 7 days for a fixed $17,000 (€15,000). One founder, one week, one product — and the repository is yours on day 8.

Talk to the person who would build it

Adrien De Coster, founder of Khufu — an AI-native product agency in Dubai. I write the code on every sprint, so the conversation about your product is with the person who builds it.

- Email: hello@khufu.io
- WhatsApp: +971 50 365 1761
- Site: https://khufu.io
- LinkedIn: https://www.linkedin.com/in/adriendecoster/
- X: https://x.com/adriendecoster_

Questions about anything in this guide are welcome even if you never hire me — the fastest way to get one answered is a direct message.